Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege minnehandteringsfeil

2.3 Containerklasser (kopiering og flytting)

2.4 Templat og abstrakte klasser

2.5 Bruk av bibliotek

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege feil

2.3 Containerklasser

2.4 Templat og abstrakte klasser

2.5 Bruk av bibliotek

INF205

26. februar 2026

Standard libraries and shared libraries

In programming with C/C++ libraries, we have already seen:

- How to work with the C standard library, *e.g.*, <cassert>, <cstring>, …
- … with the C++ standard library, *e.g.*, <iostream>, <string>, …

Technically, libraries are pre-compiled object code that can be reused.

The library needs to be accessed at three stages:

- At **compile time**, we need to **include** the **library headers**.
  - The complete source code for the libraries is unnecessary.
  - It is even possible for the library to be coded in another language.
- During **linking**, the object code is **dynamically linked** against the library.
  - At this stage, the library "**static object**" (*.so file) is needed.
  - The executable does not contain the library's object code!
- At **execution time**, the executable and the library are **loaded jointly**.
  - If the library's **static object** code is gone now, the code will not run!

Dynamic (shared) library use: Example

The **convert-to-bmp** code uses the Magick++ API of ImageMagick.[1, 2]

```
#include <Magick++.h>

int main(int argc, char** argv)
{
  // charmap input
  std::ifstream pixistrm(argv[1]);
  diskgraphics::Charmap cm;
  pixistrm >> cm;
  pixistrm.close();

  // image object setup
  Magick::InitializeMagick(*argv);
  Magick::Image img(Magick::Geometry(cm.get_sizex(), cm.get_sizey()), "white");
  img.magick("BMP");
  img.monochrome();
  img.type(Magick::BilevelType);

  // pixel-by-pixel transfer of content
  for(int x = 0; x < cm.get_sizex(); x++)
    for(int y = 0; y < cm.get_sizey(); y++)
      img.pixelColor(x, y, Magick::Color(cm.get_pixel(x, y) == 0? "black": "white"));

  // output in BMP format using one bit per pixel
  img.quantize(2);
  img.write(argv[2]);
}
```

read a "Charmap" object from a pixel graphics file, using an ad-hoc format

ImageMagick can deal with many file formats; we need an uncompressed pixel graphics format such as BMP

copy colour value of pixels

with quantize(2) we get one bit per pixel

[1]Magick++ API documentation: https://imagemagick.org/Magick++/Documentation.html

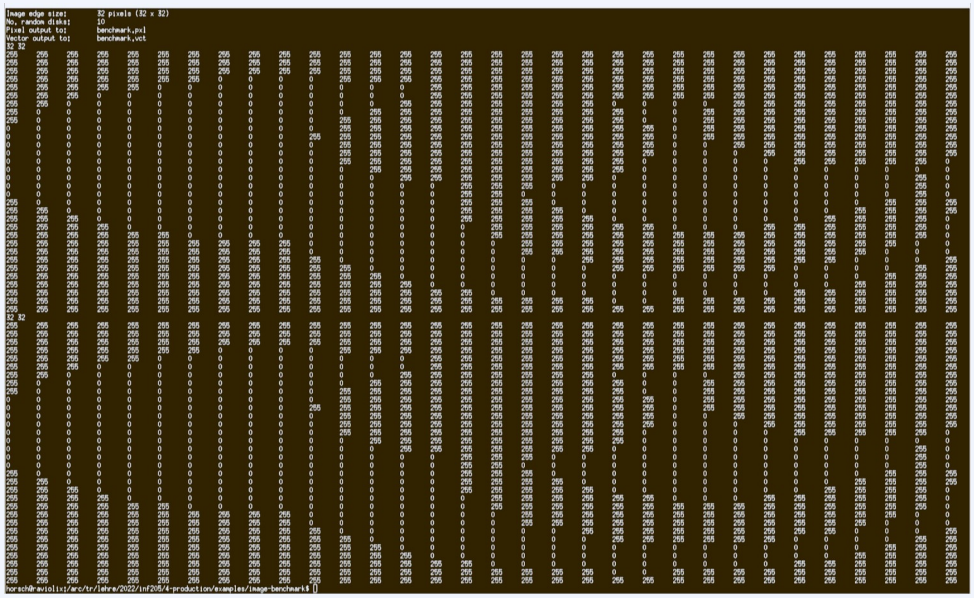
[2]Magick++ Tutorial: https://www.imagemagick.org/Magick++/tutorial/Magick++_tutorial.pdf

Dynamic (shared) library use: Example

The **convert-to-bmp** code uses the Magick++ API of ImageMagick.[1, 2]

```
#include <Magick++.h>
```

How does the compiler know where to look for this file?



[1]Magick++ API documentation: https://imagemagick.org/Magick++/Documentation.html

[2]Magick++ Tutorial: https://www.imagemagick.org/Magick++/tutorial/Magick++_tutorial.pdf

Compiling and linking with libraries

The **convert-to-bmp** code uses the Magick++ API of ImageMagick.[1, 2]

```
#include <Magick++.h>
```

How does the compiler know where to look for this file?

For this particular library, there is a tool that helps call g++ with the right flags:

```
g++ -c -std=c++17 -o <name>.o <name>.cpp `Magick++-config --cppflags`
```

-fopenmp -DMAGICKCORE_HDRI_ENABLE=1 -DMAGICKCORE_QUANTUM_DEPTH=16 **-I**/usr/local/include/ImageMagick-7

```
g++ -std=c++17 -o <name> *.o `Magick++-config --libs`
```

-L/usr/local/lib -lMagick++-7.Q16HDRI -lMagickWand-7.Q16HDRI -lMagickCore-7.Q16HDRI

More typically, you need to provide this information to the compiler by hand.

[1]Magick++ API documentation: https://imagemagick.org/Magick++/Documentation.html

[2]Magick++ Tutorial: https://www.imagemagick.org/Magick++/tutorial/Magick++_tutorial.pdf

Generating makefiles using CMake

CMake is used by many complex C/C++ projects that require developers or users to compile code on their systems, which may be very diverse. Typically:

cmake . && **make** && sudo make install

There, **CMake** generates the Makefile that is then used by **GNU make**.
We have done this before when we looked into the C++ interface to ROS.

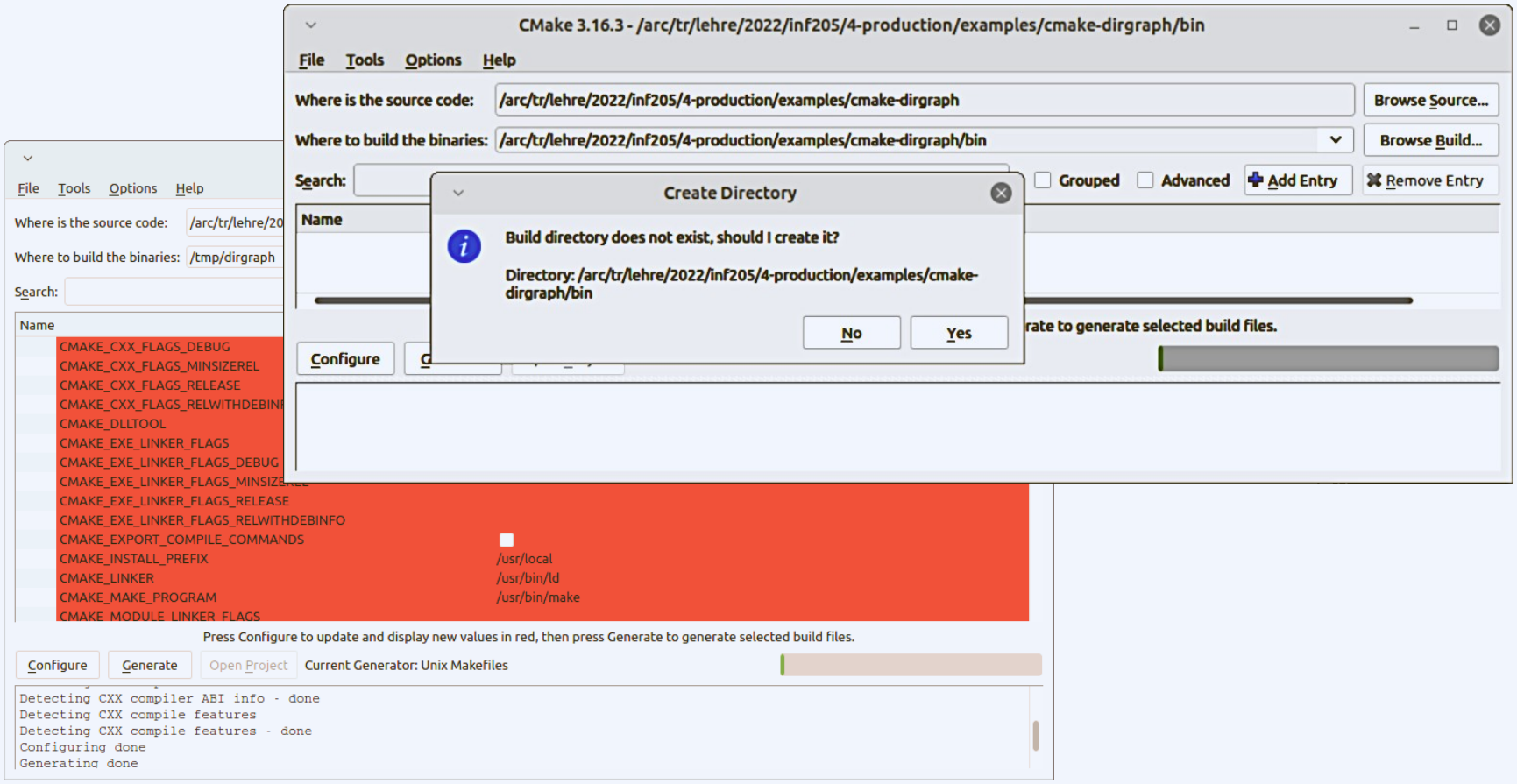
Instructions for CMake are communicated through a file called **CMakeLists.txt**.

- CMake documentation: https://cmake.org/cmake/help/latest/
- CMake tutorial: https://cmake.org/cmake/help/latest/guide/tutorial/

CMake can be helpful if your project has a complex system of dependencies, or compile-time case distinctions are needed beyond what you can implement in a simple way using GNU make; *e.g.*, embedded system cross-compiling.

CMake GUI

Graphical interface to CMake: **cmake-gui**



Creating a dynamic (shared) library

A **shared object file** can be created from an object file using g++ **-shared**:

```
g++ -fPIC -c -o first.o first.cpp
g++ -fPIC -c -o second.o second.cpp
g++ -shared -o libname.so first.o second.o
```

this is a capital i,
not a lower-case L

The library header location can be passed to g++ at compile time with -I…, and the shared object is found by the linker with the -L and -l options.

this time it is a
lower-case L

But the library also needs to be found at execution time. For that to work, it must be in the appropriate path, or one of the environment variables for library paths must be set to include the location of the shared object.

this can be
$LD_LIBRARY_PATH

Eksempel: **shared-library.zip**

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Problemliste C og oppgåve/problemliste 2

INF205

26. februar 2026

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205

Ressurseffektiv programmering

2 Minnehandtering

2.1 Kallstakken og haugen

2.2 Vanlege minnehandteringsfeil

2.3 Containerklasser (kopiering og flytting)

2.4 Templat og abstrakte klasser

2.5 Bruk av bibliotek